

Intermediate Programming Concepts

Pointers

Dr. Gang Qu

University of Maryland, College Park
gangqu@umd.edu

rvalue and lvalue

- # What is behind `int key = 5;`
 - Declare a variable `key` of integer type and give it an initial value 5.
 - 5 is the **rvalue** of `key`. Can be printed out with `%d`
 - Where is this rvalue stored? → memory
 - Where in the memory? → **lvalue**: memory address (recall: `&key` in `scanf`). Can be printed out with `%p`
 - Within the scope of the variable, its rvalue can be changed, but its lvalue remains the same.
- # Pointer is a variable that stores the memory address where another variable's rvalue is.



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

2

Pointer Declaration

Syntax: type *p_name;

```
int *p1;  
char *s1, *s2;  
int *p2, p3;
```

Then why we need
type to define a
pointer?

Assign lvalues to pointers

```
p1 = &a;    // &: address/referencing operator  
int *p = &p3;
```

Scope: the same as other variables

■ global, local, static

sizeof(): the same for all types of pointers.

■ Recall: pointer stores a memory address



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

3

Pointer Examples

```
int a, b, d=3, *pr=&d;  
a = *pr;           // a = d;  
b = *&d;           // b = d;  
*pr = a + 2;       // d = a+2;  
*pr = *pr + *pr;   // d = d+d;  
*pr = *pr * *pr;   // d = d*d;  
(*pr)++;          // d++;  
scanf("%d",pr);    // scanf("%d", &d)  
p++; p = p+2;      // p will point to new memory  
                  // locations
```

pr ↔ &d
*pr ↔ d

pointer.c



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

4

Pointer and Array

```
int a[10], b;  
int *p1, *p2;  
p1 = a;           // p1 points to a[]  
p2 = &a[0];        // same as p2 = a;  
p2 = &a[3];        // p2 points to a[3]  
                  // p2[2]  $\leftrightarrow$  a[5]
```

$a[i] \leftrightarrow *(p1+i)$
 $\leftrightarrow p1[i]$

understand the followings:

$(*p)++$; $*(p++)$; $*(++p)$; $*p++$;

pointer2.c



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

5

Pointers and Functions

Pass pointers to a function as a variable

```
int f1 (char *);  
...  
char ch, *p = &ch;  
int k = f1(p);           // or int k = f1(&ch);
```

Pass an array to a function using a pointer

```
int f2 (char *a)  
{ int i;  
  for (i=0; *(a+i)!='\0'; i++) printf("%c", *(a+i));  
  return i;  
}
```



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

6

Pointers and Functions

Return array from a function

- "cheating" way: put the return array as a parameter

```
void copy1 (char *from, char *to);
```

- Use pointer as a return type

```
char * copy2 (char *from);
```

```
char *s1, *s2, *s3;  
copy1 (s1, s2);  
s3 = copy2 (s1);
```

Returned pointer
must be static
static char *cp;

Pass variables to a function by reference

- This is the way to change the value of a variable in a function without making the variable global

```
void swap (int a, int b)  
{ int tmp;  
  tmp=a; a=b; b=tmp;  
}
```

```
void swap (int *a, int *b)  
{ int tmp;  
  tmp=*a; *a=*b; *b=tmp;  
}
```

Special Pointers

int, char, float, double, FILE

NULL pointer:

- rvalue = 0, can be any type, reserved for invalid cases
if (in_file == NULL)

void pointer:

- No type, so can be assigned to any pointer and compared with any pointer
- No pointer operations can be performed

```
void *p;
```

```
p = p1;    // ok, but the followings are all illegal
```

```
*p;        p++;        x = *(p+2);
```


Exercises

- # Explain the meaning of the following function prototypes:

```
int f1(int );  
int f2(int *);  
int f3(void);  
int *f4(void);  
int *f5(void *);  
void *f6(void);  
void f7(void *);
```



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

9

Constant Data and Pointer

- # Constant data (rvalue cannot change)

```
const int a = 5;  int b = 3;  
if (a >= b)      a++;  
b = a + 2;       a = b - 1;
```

- # Constant pointer (rvalue cannot change)

```
int x[] = {1, 2, 3, 4, 5};  
int * const p = &x[0];  
b = *p;           p++;  
b = *(p+2);       p = &x[3];  
b = p[2];         p = q;
```

pointer4.c



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

10

Pointer to Constant Data

- ✦ Pointer to constant data (pointer's rvalue can change, but not data's rvalue)

```
const int y[] = {2, 3, 5, 7};  
const int *q; q = &y[0];  
q++;           *q = 0;  
q = p;         p = q;  
q = &y[3];     q[2] = 10;
```

Question:

```
q = p;  
*q = 0;
```

- ✦ Constant pointer to constant data (none)

```
const int a = 5;  
const int *const p2 = &a;
```

pointer4.c



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

11

Exercises

```
char a, *p2;  
const char b='#', *p1;  
p1 = &b;           *p1 = '$';  
p1 = &a;  
p2 = &a;           p2 = &b;  
*p2 = '$'  
p1 = p2;           p2 = p1;
```



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

12

malloc()

Allocate free memory to a pointer

```
void *malloc(size_t size);
```

- # Parameter: size of memory in bytes needed
- # Return value: a pointer to the memory block
 - Return 0 (NULL) if there is no enough memory
- # Return type: void *, need to be casted

```
int *arr;  
arr = (int *) malloc (10 * sizeof(int));  
if (arr == NULL) //report error and exit
```



© Gang Qu (gangqu@umd.edu)

13

Dynamic Memory Allocation

- # Allocate memory

```
int *arr = malloc (10 * sizeof(int));  
if (arr == NULL) //report error and exit
```

- # Allocate cleared (with value 0) memory

```
int *arr = calloc (10, sizeof(int));
```

- # Increase or decrease memory

```
arr = (int *)realloc(arr, 11 *  
sizeof(int));
```

- # Release memory

```
free (arr); arr = NULL;
```

free () is of type void, all others are void *



© Gang Qu (gangqu@umd.edu)

14

DMA for 2-Dimensional Array

```
int **A, row, col, i, j;
// read in values of row and col
A = (int **)malloc(row*sizeof(int *));
// allocate memory for row # of pointers
// each points to one row
for (i=0; i<row; i++)
{ *(A+i) = (int *)malloc(col*sizeof(int));
// allocate memory to pointer for the ith row
  for (j=0; j<col; j++)
  { // read in A[i][j]
  }
}
```

